

Final Sprint Requirements

User can interact with the scheduler through a interactive user interface

- As a user, I want a visual dashboard so that I can navigate between creating schedules, viewing saved schedules, and exiting the program without using a command line.
- This user interface should be much easier to use than the current CLI, notably providing a better view into what tasks are currently selected
- This GUI should also provide visual feedback to the user to get a better idea of what their schedule will look like. This will involve displaying the schedule as a visual timeline rather than a list
- It should also support all of the features currently implemented in the CLI, but in an easier to use fashion
- This should include intuitive input fields in order to streamline the process of adding and modifying tasks

Input Validation

- As a user, I want the scheduler to clearly guide me when I enter invalid information so that I can understand what needs to be corrected without feeling confused or frustrated.
- This means the interface should proactively check for issues such as missing task names, negative or zero durations, invalid or reversed start/end times, and conflicting availability windows.
- When invalid inputs are detected, the GUI should not just block the user. It should highlight the problematic fields, display helpful error messages, and visually indicate what corrections are needed.

Robust Error Handling

- As a user, I want the application to gracefully handle unexpected errors so that I never lose progress or encounter crashes during schedule creation.
- If something goes wrong, the system should automatically catch the issue and provide a friendly, clear message explaining what happened.
- The GUI should offer me actionable options, such as retrying the operation, returning to the dashboard, or saving my work in a fallback mode.

Cross-Platform Stability and Installation Reliability

- As a user, I want the scheduler to run reliably on my operating system without needing complex setup steps so that I can access the tool easily and without technical frustration.

- The tool should behave the same across Windows, macOS, and Linux, meaning no odd layout issues, broken file pickers, strange fonts, or features that only work on one system but not the others.
- The installation process should also be as simple as possible, ideally providing an installer or pre-packaged application that works out of the box